# Multiple Linear Regression: Geometric Intuition and Code

## Contents


1. Overview . . . . . 2
2. From a line to a hyperplane . . . . . 3
   - 2.1 Two inputs: a plane . . . . . 3
   - 2.2 More inputs: a hyperplane . . . . . 4
3. The equation . . . . . 5
   - 3.1 What the coefficients mean . . . . . 6
4. Multiple linear regression in scikit-learn . . . . . 7
5. Summary . . . . . 9
6. Sources . . . . . 9
7. Key terms . . . . . 9

**Where this fits:** Pipeline map step 8 (Model). See the Course map.

| | | | | | | |
|---|---|---|---|---|---|---|
| 0 Foundations | 1 Frame | 2 Get data | 3 Understand | 4 Clean | 5 Features | 6 Reduce |
| 7 Split | **8 Model** | 9 Evaluate | 10 Tune | 11 Deploy | 12 Test | 13 Monitor |

- **Builds on:** Regression problems; Simple linear regression.
- **Leads to:** Normal equation; Batch gradient descent; Polynomial regression; Ridge regression.
- **Compare with:** ANN for regression.

## 1. Overview

**Key point:** Multiple linear regression is simple linear regression with more than one feature. With two inputs it fits a **plane** (G-1502); with more, a hyperplane.

A **feature** (G-772) is an input variable (one column of the data table), the **target** (G-1949) is the output we predict, and an **observation** (G-1374) is one record (one row). Figure 1 marks all three on the first rows of this Note's example data.

| row | feature 1 ($x_1$) | feature 2 ($x_2$) | target (y) |
|---|---|---|---|
| 1 | 0.27 | -0.52 | 32.6 |
| 2 | -1.43 | 0.5 | -75.3 |
| 3 | -0.1 | 1.7 | 56.2 |
| 4 | -0.28 | -1.54 | -97 |
| 5 | -0.19 | 0.7 | -103.4 |

green row: one observation (one student, one house, one record)

**Figure 1:** The first five observations of the example data. Blue columns: the two features. Orange column: the target. Green row: one observation.

**Simple linear regression** (G-1808) used one feature, CGPA, to predict the package (a student's salary offer) with a best-fit line. Real data almost always has several features: CGPA, gender, IQ, 12th-grade marks, and so on. Linear regression with more than one feature is called **multiple linear regression** (G-1279).

Nothing new has to be learned for multiple linear regression: everything from simple linear regression carries over. **Simple linear regression** (G-1808) is just the special case with one feature. A recipe works the same way: the cake's taste depends on sugar, flour and butter together, each in its own amount, instead of on sugar alone. This Note covers the geometry and the code. The mathematics is derived in the model in matrix form, and the model is coded from scratch in our own class.

## 2. From a line to a hyperplane

> **Key point:** One input: a line in 2D. Two inputs: a **plane** (G-1502) in 3D. n inputs: a hyperplane in n + 1 dimensions.

### 2.1 Two inputs: a plane

> **Key point:** With CGPA and IQ as inputs, the data lives in 3D, and the model is a flat plane through the cloud of points.

Take two inputs, CGPA ($x_1$) and IQ ($x_2$), and the package ($y$) as output. Each student is now a point in 3D: CGPA along one axis, IQ along another, package upwards.

In 2D we drew the line that passes closest to all points. In 3D we draw a flat **plane** (G-1502) that cuts through the cloud of points: some points lie above it, some below, and the plane keeps as close as possible to all of them (Figure 2).

Figure 2 and the figures after it use this Note's example data of Figure 1: 100 made-up **observations** (G-1374) with two features, feature1 and feature2, standing in for CGPA and IQ (section 4 shows how the data is made).

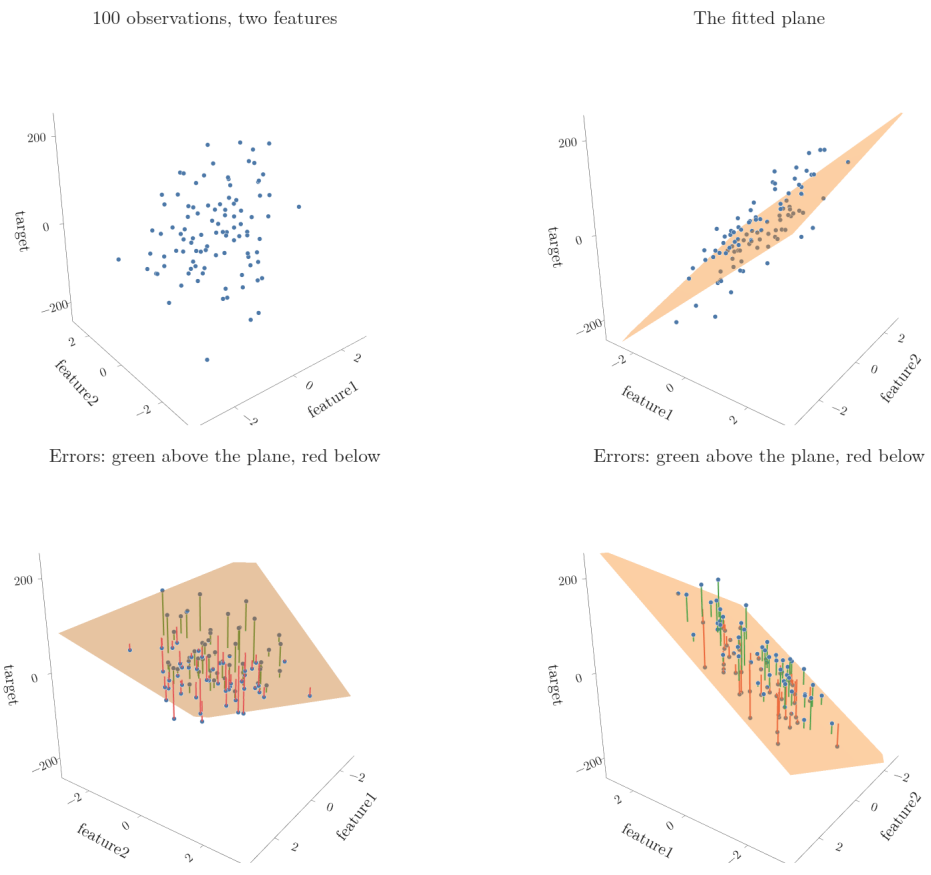



**Figure 2:** 100 observations in 3D, then the fitted plane, then each observation's error as a stick to the plane (green above, red below), while the view circles once

Watch the sticks as the view turns: about half the points sit above the plane (50 of 100) and half below, and from the side the plane cuts through the middle of the cloud. Fitting the plane means making these sticks, squared and added up, as small as possible, just as with the line.

### 2.2 More inputs: a hyperplane

> **Key point:** Beyond three dimensions we cannot draw the picture, but the equation keeps the same form.

With three inputs the data is 4-dimensional, and the model is the 4D version of a **plane** (G-1502). A flat surface in more than three dimensions is called a **hyperplane** (G-911). We cannot draw it, but the mathematics works exactly the same (Figure 3). A small case with three inputs and made-up numbers:

$$\hat{y} = 2 + 3x_1 + 1x_2 - 0.5x_3$$

Take a point with $x_1 = 1$, $x_2 = 2$ and $x_3 = 4$:

$$3 \times 1 = 3$$
$$1 \times 2 = 2$$
$$-0.5 \times 4 = -2$$

$$\hat{y} = 2 + 3 + 2 - 2 = 5$$

The steps are the same as for two inputs; only one more term is added.

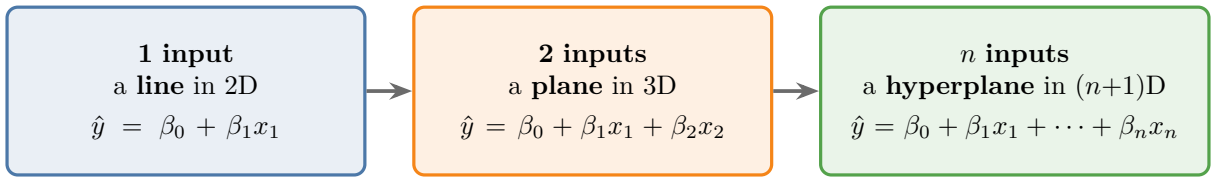



**Figure 3:** From a line to a hyperplane

## 3. The equation

> **Key point:** The prediction is the intercept plus one coefficient times each input:
>
> $$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n$$

For **simple linear regression** (G-1808) we wrote $y = mx + b$. With more inputs, letters run out, so we rename the numbers with the Greek letter beta: $\beta_0$ for the intercept and $\beta_1, \beta_2, \ldots$ for the slopes.

In words: start from the intercept, then add each input multiplied by its own coefficient.

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

The symbols: $\hat{y}$ ("y hat") is the predicted **target** (G-1949); $x_1, x_2, \ldots, x_n$ are the $n$ inputs, where the small number labels which input (for the placement data, $x_1$ could be CGPA and $x_2$ IQ); and $\beta_0, \beta_1, \ldots, \beta_n$ are the $n + 1$ numbers that the model learns.

With numbers, from the model in Figure 2:

$$\hat{y} = -1.9 + 58.6 x_1 + 29.1 x_2$$

For a point with $x_1 = 1$ and $x_2 = 0.5$:

$$\hat{y} = -1.9 + 58.6 \times 1 + 29.1 \times 0.5$$

$$\hat{y} = -1.9 + 58.6 + 14.6 = 71.3$$

Figure 4 builds this prediction one term at a time: start at the intercept, then let each **feature** (G-772) add its own share.

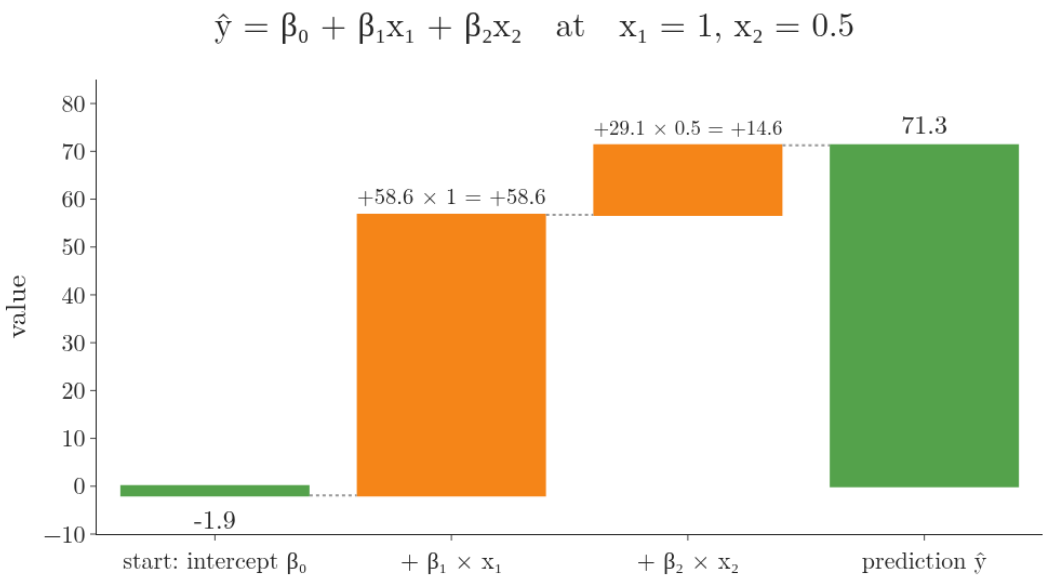



**Figure 4:** One prediction built term by term: the intercept $-1.9$, plus $58.6 \times 1$, plus $29.1 \times 0.5$, gives 71.3.

With $n$ features there are $n + 1$ numbers to find: one **coefficient** (G-407) per feature plus the **intercept** (G-960). Training the model means finding them.

### 3.1 What the coefficients mean

> **Key point:** Each **coefficient** (G-407) is the change in the output when its feature rises by 1 and the other features stay the same: the feature's weight.

The meaning carries over from the slope $m$ of **simple linear regression** (G-1808):

- $\beta_1$: how much the prediction changes when $x_1$ rises by 1 and every other input stays fixed.
- $\beta_2$: the same for $x_2$, and so on.
- $\beta_0$: the prediction when every input is 0.

Figure 5 slices the **plane** (G-1502) of Figure 2 at three fixed values of feature2. Each slice is a line with slope 58.6, and the slices sit 29.1 apart: one coefficient per direction.

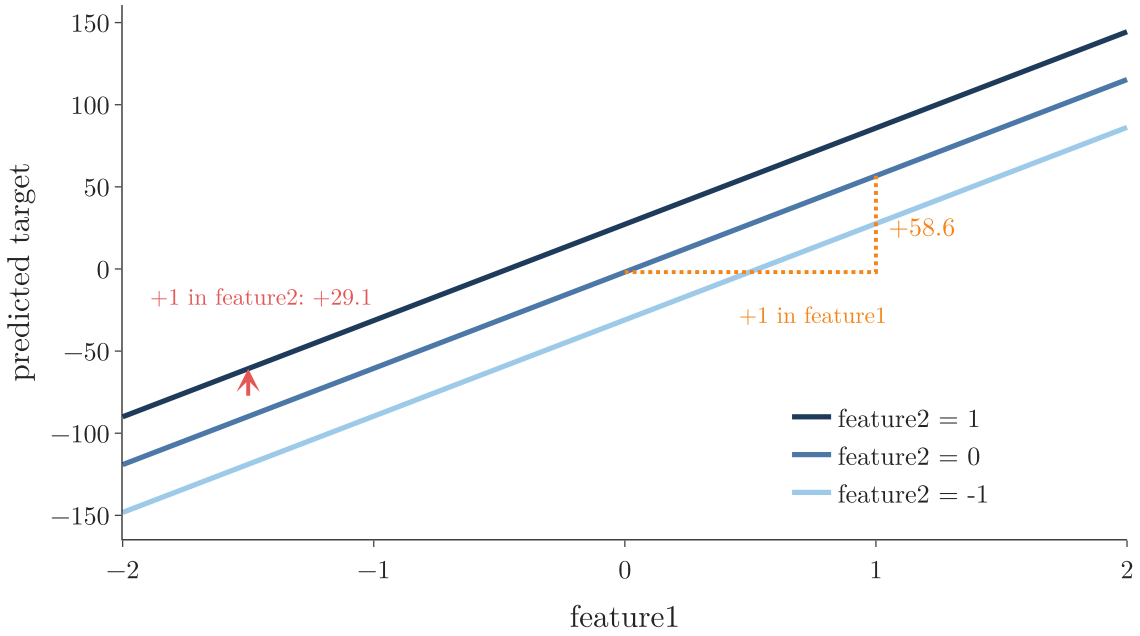



**Figure 5:** Slices of the fitted plane at feature2 = −1, 0 and 1. One step along feature1 adds 58.6 (orange); one step in feature2 lifts the whole line by 29.1 (red)

So the coefficients act as **weights** (G-2111): in Figure 2, the **target** (G-1949) depends about twice as strongly on feature 1 (58.6 per unit) as on feature 2 (29.1 per unit). In the placement example, $\beta_1$ would say how much the package depends on CGPA and $\beta_2$ how much on IQ. A coefficient near 0 means the target hardly depends on that **feature** (G-772).

> **Extra:** Comparing coefficients only makes sense when the inputs are on similar scales. A coefficient of 58.6 per unit of CGPA and 0.05 per IQ point says nothing about which matters more, because one IQ point is a much smaller step than one CGPA point. **Standardising** (G-1874) the inputs first puts all coefficients on the same footing (Gelman 2008). In Figure 2 the comparison is fair: both inputs from `make_regression` already have a standard deviation (typical distance from the mean) of about 1.

## 4. Multiple linear regression in scikit-learn

> **Key point:** The code is exactly the same as for one feature; LinearRegression simply returns one **coefficient** (G-407) per feature.

The example data has 100 **observations** (G-1374), 2 features and some noise, made with scikit-learn's `make_regression`.

> **Python:** Training on two **features** (G-772).

```
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
```

```
from sklearn.model_selection import train_test_split

X, y = make_regression(n_samples=100, n_features=2,
    n_informative=2, noise=50, random_state=7)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=3)

lr = LinearRegression().fit(X_train, y_train)
lr.coef_          # [58.6, 29.1]: one per feature
lr.intercept_     # -1.9
```

`make_regression` (G-1153) invents data that follows a linear pattern plus random noise (`noise=50` sets how big the random scatter is); `random_state` fixes it so the numbers repeat.

The metrics (MAE, the average size of the miss; MSE, the average squared miss; $R^2$, the share of the target's spread that the model explains) are computed exactly as before: the errors are now gaps to the **plane** (G-1502) instead of gaps to the line. With several features, adjusted $R^2$ is the fairer score, because it charges a penalty for each feature.

On the 20 test observations: MAE 40.1, MSE 2614.9 and $R^2 = 0.61$. The noise added to the data limits how good any plane can be: with less noise the same code scores higher, and with no noise it fits perfectly ($R^2$ of 0.85 at `noise=25`, 1.0 at `noise=0`; see the Notebook). Figure 6 shows the test predictions for all three: as the noise falls, the points close in on the diagonal of perfect predictions.

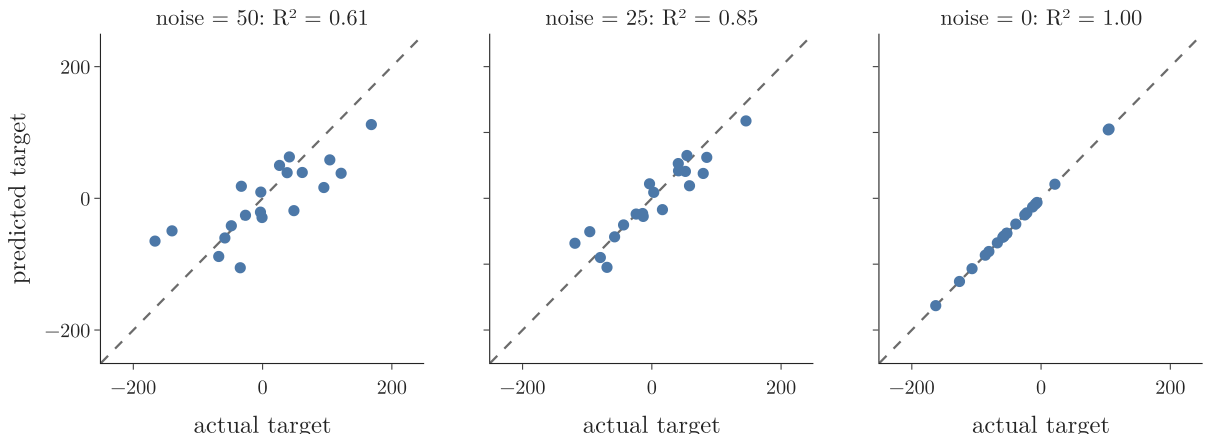



**Figure 6:** Predicted against actual target on the 20 test observations, for the same recipe with noise 50, 25 and 0

**Extra:** To draw the plane, we predict on a grid of $(x_1, x_2)$ points and plot the predictions as a surface. The Notebook does this with Plotly, and the 3D plot can be turned with the mouse to see the points above and below the plane.

## 5. Summary

| | Simple linear regression | Multiple linear regression |
|---|---|---|
| Inputs | 1 | 2 or more |
| Shape of the model | line | plane (2 inputs), hyperplane (more) |
| Equation | $\hat{y} = mx + b$ | $\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n$ |
| Numbers to find | 2 | $n + 1$ |
| scikit-learn | `LinearRegression` | the same `LinearRegression` |

- The two columns differ only in the number of inputs, so everything learned for **simple linear regression** (G-1808) (least squares, the metrics, the code) carries over unchanged.
- Real data almost always has several inputs, so **multiple linear regression** (G-1279) is the version used in practice.
- Each coefficient is the change in the output per unit of its input, with the other inputs fixed: a weight, so the coefficients show which input the **target** (G-1949) depends on most (58.6 against 29.1 here), but only when the inputs are on similar scales.
- The **intercept** (G-960) $\beta_0$ is the prediction when every input is 0, so it sets the height of the **plane** (G-1502).
- Together these answer the opening question: multiple linear regression is simple linear regression with more inputs, fitting a plane or **hyperplane** (G-911) instead of a line.

## 6. Sources

### Built from

- CampusX, "Multiple Linear Regression | Geometric Intuition & Code", YouTube, https://www.youtube.com/watch?v=ashGekqstl8
- StatQuest with Josh Starmer, "Multiple Regression, Clearly Explained!!!", YouTube, https://www.youtube.com/watch?v=EkAQAi3a4js ($R^2$ is computed as in simple regression; adjusted $R^2$ for the extra parameters)

### Other references


- Gelman, A. (2008). Scaling regression inputs by dividing by two standard deviations. *Statistics in Medicine* 27(15): 2865–2873.


## 7. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

| Term | Meaning |
|---|---|
| Plane (G-1502) | A flat surface in 3D; the model for two input columns. |
| Multiple linear regression (G-1279) | Linear regression with several input columns. |
| Hyperplane (G-911) | A flat surface in more than three dimensions; the model for three or more input columns. |
| Coefficient ($\beta_i$) (G-407) | The weight of one input column: the change in the output per unit of that input, others fixed. |
| make_regression (G-1153) | scikit-learn function that generates data following a linear pattern plus noise. |
| Feature (G-772) | One piece of information about each example that a model uses (e.g. a student's CGPA). |
| Target (G-1949) | The output column we predict, such as the class; also called the label. |
| Observation (G-1374) | One record of a dataset: one row of the data table. |
| Simple linear regression (G-1808) | Linear regression with one input column: it fits a straight line $y = mx + b$ to predict the output from that one feature. |
| Intercept (G-960) | The line's value when the input is 0; $b$ in $y = mx + b$. |
| Weight (in a weighted mean) (G-2111) | A number saying how much a value counts in a weighted mean. |
| Standardization (G-1874) | Scaling a column to mean 0 and standard deviation 1 (subtract the mean, divide by the standard deviation), so features measured on different scales become comparable. |